Using LLM and Agentic AI to Build the Quant Analysis Platform

A Practical Learning Guide for Students

---

1. Introduction — What This Guide Covers

This guide teaches you how to **systematically use LLMs (Large Language Models) and Agentic AI** to develop, enhance, and maintain the Quant Analysis Platform — a Flask-based financial analysis application that uses Google Gemini for news sentiment analysis, data visualization, and report generation.

You will learn:

- How LLMs work and how to call them via API
- Prompt engineering techniques for structured, reliable output
- Agentic AI concepts and how they apply to your development workflow
- How to use free/cheap AI tools as a student
- Hands-on examples specific to this project

---

2. Free and Cheapest LLM Tools for Students (March 2026)

2.1 Google Gemini API Free Tier (Recommended — Already in Your Project)

Your project already uses `gemini-2.0-flash-lite`. Google's free tier is the best starting point:

- **No credit card required** — sign up at aistudio.google.com
- **Models available for free**:
  - Gemini 2.5 Pro — 5 requests/min, 100 requests/day (best reasoning)
  - Gemini 2.5 Flash — 10 requests/min, 250 requests/day (good balance)
  - Gemini 2.5 Flash-Lite — 15 requests/min, 1,000 requests/day (highest throughput)
- **1 million token context window** on all models
- **Setup**: Get your API key → put it in your `.env` file as `GOOGLE_API_KEY`

Official docs: ai.google.dev/gemini-api/docs/pricing

2.2 Other Free Options

Tool	Free Tier	Best For	Link
Groq	Free tier with Llama 3 models	Ultra-fast inference	console.groq.com
OpenRouter	20 req/min, 200 req/day on free models	Trying multiple models via one API	openrouter.ai
GitHub Models	Free with GitHub PAT (Personal Access Token)	Experimenting with various models	github.com/marketplace?type=models
Hugging Face Inference	Free for models < 10GB	Open-source model experimentation	huggingface.co/inference-api
Ollama (Local)	Completely free, runs on your machine	Offline development, no API limits	ollama.com

2.3 Agentic AI Coding Tools

Tool	Cost	What It Does
Claude.ai (Free tier)	Free	Chat-based AI for planning, code review, writing
Claude Code	$20/month (Pro plan required)	Terminal-based agentic coding assistant
Cursor IDE	Free tier (limited)	AI-powered code editor with Claude/GPT models
GitHub Copilot	Free for students via GitHub Education	Code completion and chat in VS Code
Gemini CLI	Free	Google's CLI coding tool

> **Recommendation for students**: Use the **Gemini API free tier** for your project's LLM features, **Claude.ai free tier** or **GitHub Copilot (free for students)** for coding assistance, and **Ollama** if you want to experiment locally with no limits.

---

3. Understanding LLMs — The Basics

3.1 What Is an LLM?

A Large Language Model is a neural network trained on massive text data that can generate human-like text, analyze content, and follow instructions. In your project, the LLM (Gemini) reads news articles and produces

structured sentiment analysis.

3.2 How Your Project Uses LLMs

Your project's LLM integration lives in `backend/services/llm_service.py`. Here is the flow:

> **User requests sentiment analysis for a company** ↓ **Backend fetches recent news articles** (Guardian API + yfinance) ↓ **News articles are formatted** into text ↓ **Text + Prompt are sent** to Gemini API ↓ **Gemini returns** structured JSON response ↓ **Backend parses response** and stores/displays it

3.3 Key Concepts

Tokens: The basic units LLMs process. Roughly 1 token ≈ 0.75 words. You are charged per token (input + output).

Context Window: How much text the model can "see" at once. Gemini's free tier offers 1 million tokens — enough to process dozens of news articles simultaneously.

Temperature: Controls randomness. Lower (0.0–0.3) = more deterministic and consistent. Higher (0.7–1.0) = more creative. For sentiment analysis, use **low temperature** (0.1–0.3).

Structured Output: Getting the LLM to return data in a specific format (like JSON) so your code can parse it reliably.

---

4. Prompt Engineering — Getting Reliable Results

Prompt engineering is the skill of writing instructions that make LLMs produce accurate, consistent, and useful output. This is the **most important skill** for working with LLMs in your project.

4.1 The 4-Block Prompt Pattern

Structure every important prompt with these four sections:

> **ROLE:** Who the LLM should act as **CONTEXT:** Background information it needs **TASK:** Exactly what to do **OUTPUT FORMAT:** The exact structure of the response

Example for your project's sentiment analysis:

```python
```

```
prompt = """
ROLE: You are a senior financial analyst specializing in market sentiment.

CONTEXT: You are analyzing recent news articles about {company_name} ({ticker}).
The following articles were published in the last 7 days.

TASK: Analyze the overall sentiment of these news articles and assess their
potential impact on the company's stock price.

OUTPUT FORMAT: Return ONLY a valid JSON object with this exact structure:
{
  "overall_news_summary": "2-3 sentence summary of key themes",
  "brief_overall_sentiment": "Positive|Negative|Neutral|Mixed (Score: X/100)",
  "reasons_for_sentiment": "Detailed explanation of why",
  "market_outlook": "Short-term market outlook based on news",
  "detailed_explanation": "Comprehensive analysis paragraph",
  "key_offerings": ["list", "of", "key", "topics"],
  "financial_dates": ["any upcoming dates mentioned"]
}

Do not include any text before or after the JSON.

NEWS ARTICLES:
{news_text}
"""
```

4.2 Key Prompt Engineering Techniques

1. Be Specific, Not Vague

> ❌ **Bad:** "Analyze this news"
> ✅ **Good:** "Analyze the sentiment of these 5 news articles about Apple Inc. Return a JSON with overall_sentiment, confidence_score (0-100), and top_3_reasons as an array of strings."

2. Use Few-Shot Examples

Show the LLM an example of the output you want:

```python
```

```
prompt = """
Analyze news sentiment. Here is an example:

Input: "Company X reports record profits, stock surges 15%"
Output: {"sentiment": "Positive", "confidence": 92, "reason": "Record profits and stock surge"}

Now analyze this:
Input: "{actual_news_text}"
Output:
"""
```

3. Constrain the Output

> Return ONLY one of: "Positive", "Negative", "Neutral", "Mixed"
> Do not explain your choice.

4. Request Self-Validation

> Before returning your response, verify:
> - Is the JSON valid?
> - Are all required fields present?
> - Is the sentiment score between 0 and 100?

5. Handle Uncertainty Explicitly

> If the news articles are insufficient to determine sentiment,
> set sentiment to "Neutral" and confidence to 50.
> Do not guess or fabricate information.

4.3 Validating LLM Output in Code

Never trust LLM output blindly. Always validate:

```python
```

```python
import json
import re

def parse_llm_response(raw_text):
    """Safely parse LLM response, extracting JSON."""
    # Try to find JSON in the response
    json_match = re.search(r'```json\s*(\{.*?\})\s*```', raw_text, re.DOTALL)
    if not json_match:
        json_match = re.search(r'(\{.*\})', raw_text, re.DOTALL)

    if json_match:
        try:
            data = json.loads(json_match.group(1))
            # Validate required fields
            required = ["overall_news_summary", "brief_overall_sentiment"]
            for field in required:
                if field not in data:
                    data[field] = "Not available"
            return data
        except json.JSONDecodeError:
            pass

    # Fallback: return a safe default
    return {
        "overall_news_summary": "Analysis could not be parsed.",
        "brief_overall_sentiment": "Neutral (Score: 50/100)",
        "reasons_for_sentiment": raw_text[:500]
    }
```

This pattern is already used in your project's `llm_service.py` — study it carefully.

4.4 Useful Prompt Engineering Resources

- **Anthropic's Prompt Engineering Guide**: docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
- **Google's Prompt Engineering for Gemini**: ai.google.dev/gemini-api/docs/prompting-strategies
- **Prompt Engineering Guide (Community)**: promptingguide.ai
- **Lakera Prompt Engineering Guide 2026**: lakera.ai/blog/prompt-engineering-guide

---

5. Agentic AI — From Chatbot to Autonomous System

5.1 What Is Agentic AI?

Traditional LLM usage is a single question → single answer. **Agentic AI** goes further: the AI can **plan, use tools, take actions, evaluate results, and iterate** until a goal is achieved.

Think of it as the difference between:

- **Chatbot**: "What's the sentiment of this article?" → answer
- **Agent**: "Monitor Apple's news daily, analyze sentiment, compare to historical trends, and alert me if sentiment drops below 30/100" → the AI plans the steps, fetches data, calls APIs, runs analysis, and sends the alert

5.2 The Five Core Agentic Patterns

These are the fundamental design patterns used in agentic AI systems:

1. Tool Use — The agent calls external tools (APIs, databases, calculators)

> **Agent thinks:** "I need Apple's latest news" **Agent action:** Call Guardian API → get articles **Agent thinks:** "Now I need to analyze sentiment" **Agent action:** Call Gemini API with articles → get sentiment **Agent thinks:** "I should store this result" **Agent action:** Insert into MySQL database

2. ReAct (Reason + Act) — The agent alternates between reasoning and taking action

> **Thought:** I need to check if Apple's stock dropped today **Action:** Fetch stock data from yfinance **Observation:** Apple dropped 3.2% **Thought:** I should check if there's negative news **Action:** Fetch news from Guardian API **Observation:** 3 articles about regulatory concerns **Thought:** I should analyze these for sentiment **Action:** Call Gemini API for sentiment analysis **Result:** Negative sentiment (Score: 25/100) due to regulatory concerns

3. Reflection — The agent reviews and improves its own output

> **First draft:** Sentiment analysis produced **Self-review:** "The confidence score seems too high given mixed signals" **Revised output:** Adjusted confidence from 85 to 65

4. Planning — The agent breaks a complex task into steps before executing

> **Goal:** Generate a full investment report for Tesla **Plan:**
>
> 1. Fetch financial data (yfinance)
> 2. Fetch recent news (Guardian API)
> 3. Analyze news sentiment (Gemini)
> 4. Generate financial trend analysis
> 5. Compile everything into report format

> 6. Store report in database
>
> **Execute each step...**

5. Multi-Agent Collaboration — Multiple specialized agents work together

> **Agent 1 (Data Collector):** Fetches financial data + news **Agent 2 (Analyst):** Performs sentiment analysis **Agent 3 (Report Writer):** Compiles findings into a report **Orchestrator:** Coordinates the agents and merges results

5.3 How Your Project Already Uses Agentic Patterns

Your Quant Analysis Platform already implements several agentic patterns:

Pattern	Where in Your Project
Tool Use	`llm_service.py` calls Gemini API; `data_service.py` calls yfinance and Guardian API
Planning	The scheduled `daily_news_update` task in `tasks.py` follows a planned sequence
Structured Output	Prompts request JSON format; code validates and parses the response

5.4 Where You Can Add More Agentic Behavior

Here are practical ways to make your project more agentic:

A. Multi-Step Analysis Chain

Instead of one LLM call, chain multiple calls:

```python
```

```python
# Step 1: Summarize each news article individually
summaries = []
for article in articles:
    summary = call_gemini("Summarize this article in 2 sentences: " + article['text'])
    summaries.append(summary)

# Step 2: Analyze overall sentiment from summaries
overall = call_gemini(f"""
    Given these {len(summaries)} article summaries about {company},
    provide an overall sentiment analysis...
    Summaries: {summaries}
""")

# Step 3: Self-validate
validation = call_gemini(f"""
    Review this sentiment analysis for accuracy and consistency.
    If you find issues, provide a corrected version.
    Analysis: {overall}
""")
```

B. Automated Prompt Improvement (Reflection Pattern)

Your project has a prompt management system (`prompt_versions` table). You could add:

```python
def auto_improve_prompt(current_prompt, sample_output, expected_quality):
    """Use LLM to improve its own prompt."""
    improvement_prompt = f"""
    You are a prompt engineer. Review this prompt and its output.

    CURRENT PROMPT: {current_prompt}
    SAMPLE OUTPUT: {sample_output}
    QUALITY ISSUES: {expected_quality}

    Suggest an improved version of the prompt that would produce
    better, more consistent results. Return only the improved prompt.
    """
    return call_gemini(improvement_prompt)
```

C. Intelligent Alert System (ReAct Pattern)

```python
```

```python
def smart_alert_check(company_id):
    """Agent that reasons about whether to trigger an alert."""
    # Reason: What data do I need?
    financial_data = get_financial_data(company_id)
    news = get_recent_news(company_id)

    # Act: Analyze the situation
    analysis = call_gemini(f"""
        Given this financial data and news for {company_id},
        should we alert the user? Consider:
        1. Price changes > 5%
        2. Negative sentiment in multiple articles
        3. Unusual trading volume

        Financial data: {financial_data}
        News headlines: {[n['title'] for n in news]}

        Return JSON: {{"should_alert": true/false, "reason": "...", "severity": "low/medium/high"}}
    """)

    # Act on the result
    if analysis['should_alert']:
        trigger_alert(company_id, analysis['reason'], analysis['severity'])
```

---

6. Systematic Development Workflow Using AI

6.1 The AI-Assisted Development Cycle

Here is a step-by-step workflow for using AI to build features in your project:

> **Step 1: PLAN with AI** → Describe the feature to Claude/ChatGPT → Ask it to break down into tasks and identify files to modify → Review and refine the plan
> **Step 2: DESIGN with AI** → Ask AI to suggest the database schema changes → Ask AI to draft the API endpoint design → Ask AI to outline the frontend changes
> **Step 3: IMPLEMENT with AI assistance** → Use GitHub Copilot / Cursor for code completion → Use Claude/ChatGPT to explain unfamiliar code patterns → Write code yourself, using AI to accelerate, not replace
> **Step 4: TEST with AI** → Ask AI to generate test cases for your code → Use AI to review your code for bugs and security issues → Run tests and fix issues
> **Step 5: REVIEW with AI** → Paste your code into Claude/ChatGPT for code review → Ask it to check for edge cases, error handling, and best practices → Iterate based on feedback

6.2 Example: Adding a New Feature Using This Workflow

Feature: Add predictive financial trends (the `predict_financial_trends` placeholder in `data_service.py`)

Step 1 — Plan (Ask Claude or ChatGPT):

> I'm working on a Flask financial analysis platform. I need to implement a predict_financial_trends function in data_service.py.
> The function should:
> - Take a company_id and timeframe
> - Fetch historical financial data from the database
> - Use simple statistical methods (moving averages, linear regression)
> - Optionally use Gemini LLM to provide a narrative explanation
> - Return predictions with confidence intervals
>
> What steps should I follow? What files need to change?

Step 2 — Design (Continue the conversation):

> Based on your plan, please design:
> 1. The function signature and return type
> 2. The API endpoint in data_routes.py
> 3. The frontend display approach
>
> Keep it simple — we're using Flask, jQuery, and Chart.js.

Step 3 — Implement (Use AI to help write code, but understand every line):

> Write the predict_financial_trends function.
> Use numpy for linear regression and pandas for moving averages.
> Include error handling for insufficient data.
> Follow the existing code style in data_service.py.

Step 4 — Test (Ask AI to generate tests):

> Generate pytest test cases for predict_financial_trends.
> Include tests for: normal case, insufficient data, invalid company_id,
> and edge cases with flat/volatile data.
> Follow the existing test patterns in test_data_service.py.

6.3 Using AI Chat Tools Effectively

Do:

- Give full context (paste relevant code, describe the project structure)
- Ask specific questions ("How should I handle the case where Gemini returns invalid JSON?")

- Iterate — refine your request based on the response
- Verify AI-generated code before using it
- Use AI to explain code you don't understand

Don't:

- Blindly copy-paste without understanding
- Skip testing because "the AI wrote it"
- Assume AI output is always correct
- Use AI to bypass learning — use it to accelerate learning

---

7. Hands-On Exercises

Exercise 1: Improve the Sentiment Analysis Prompt

1. Open `backend/services/llm_service.py`
2. Find the default prompt in `analyze_news_sentiment_gemini`
3. Rewrite the prompt using the 4-Block Pattern (Role, Context, Task, Output Format)
4. Add a few-shot example
5. Test with real news data and compare output quality

Exercise 2: Build a Simple Agentic Chain

1. Create a new function `comprehensive_company_analysis(company_id)` that:
    - Fetches financial data
    - Fetches news
    - Calls Gemini for sentiment analysis
    - Calls Gemini again to generate an investment recommendation
    - Combines everything into a structured report
2. Each step should check the previous step's output before proceeding
3. Add error handling at each step

Exercise 3: Implement the Reflection Pattern

1. After your sentiment analysis returns a result, add a second LLM call that:
    - Reviews the analysis for consistency

- Checks if the sentiment score matches the written explanation
- Returns a "confidence" rating for the analysis quality

2. If confidence is below 60%, retry the original analysis with a refined prompt

Exercise 4: Compare LLM Providers

1. Sign up for free tiers on Gemini AND one other provider (Groq, OpenRouter)
2. Send the same news articles and prompt to both
3. Compare: response quality, speed, consistency, and JSON formatting
4. Document your findings

Exercise 5: Prompt Version Management

1. Use the existing prompt management system (`/prompts` endpoints)
2. Create 3 different versions of the sentiment analysis prompt
3. Run each version against the same set of articles
4. Evaluate which version produces the most accurate and consistent results
5. Activate the best-performing version

---

8. Key Agentic AI Frameworks and Tools to Explore

If you want to go deeper into building agentic systems, these are the key frameworks:

Framework	Language	Best For	Link
LangChain	Python	General LLM application building	langchain.com
LangGraph	Python	Complex agent workflows with state	langchain-ai.github.io/langgraph
CrewAI	Python	Multi-agent collaboration	crewai.com
AutoGen	Python	Multi-agent conversations	github.com/microsoft/autogen
Ollama	Any	Running models locally for free	ollama.com

For this project, **you don't need a framework** — direct API calls to Gemini (as already implemented) are sufficient. Frameworks become valuable when building more complex multi-agent systems.

---

9. Useful Learning Resources

Prompt Engineering

- Anthropic's Prompt Engineering Docs: docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
- Google Gemini Prompting Guide: ai.google.dev/gemini-api/docs/prompting-strategies
- Prompt Engineering Guide (Community): promptingguide.ai
- Analytics Vidhya — Master Prompt Engineering 2026: analyticsvidhya.com/blog/2026/01/master-prompt-engineering

Agentic AI

- The 2026 Guide to Agentic Workflow Architectures: stackai.com/blog/the-2026-guide-to-agentic-workflow-architectures
- Machine Learning Mastery — 7 Agentic AI Trends 2026: machinelearningmastery.com/7-agentic-ai-trends-to-watch-in-2026
- Deloitte — Agentic AI Strategy: deloitte.com/us/en/insights/topics/technology-management/tech-trends/2026/agentic-ai-strategy.html

Free LLM APIs

- Free LLM API Directory (45+ providers): free-llm.com
- GitHub — Free LLM API Resources List: github.com/cheahjs/free-llm-api-resources
- Google AI Studio (free Gemini access): aistudio.google.com

AI-Assisted Coding

- Claude Code Documentation: code.claude.com/docs/en/overview
- Coursera — Claude Code Software Engineering (free to audit): coursera.org/learn/claude-code
- GitHub Copilot (free for students): education.github.com

Google Gemini API

- Gemini API Quickstart: ai.google.dev/gemini-api/docs/quickstart
- Gemini API Pricing & Free Tier: ai.google.dev/gemini-api/docs/pricing
- Python SDK (`google-generativeai`): pypi.org/project/google-generativeai

---

10. Quick Reference — Gemini API Code Snippet

Here is the minimal code to call Gemini in Python (as used in your project):

```python
import google.generativeai as genai
import os
import json

# Configure API key
genai.configure(api_key=os.environ.get('GOOGLE_API_KEY'))

# Initialize model
model = genai.GenerativeModel('gemini-2.0-flash-lite')

# Define your prompt
prompt = """
You are a financial analyst. Analyze the sentiment of these news articles.
Return a JSON object with "sentiment" (Positive/Negative/Neutral)
and "confidence" (0-100).

Articles:
- "Apple reports record Q4 earnings, beating analyst expectations"
- "New iPhone sales exceed projections by 20%"
"""

# Call the API
response = model.generate_content(prompt)

# Parse the response
raw_text = response.text.strip()
try:
    result = json.loads(raw_text)
    print(f"Sentiment: {result['sentiment']}, Confidence: {result['confidence']}")
except json.JSONDecodeError:
    print(f"Could not parse JSON. Raw response: {raw_text}")
```

---

11. Summary — Your Learning Path

> **Week 1:** Understand how LLMs work + Set up Gemini free tier → Read Section 3, try the code in Section 10

> **Week 2:** Master prompt engineering → Read Section 4, complete Exercise 1
> **Week 3:** Understand agentic patterns + Apply to your project → Read Section 5, complete Exercises 2 and 3
> **Week 4:** Use AI tools in your development workflow → Read Section 6, complete Exercises 4 and 5
> **Ongoing:** Explore frameworks (Section 8) and resources (Section 9) as you build more advanced features

Remember: AI is a tool to accelerate your learning and development, not a replacement for understanding. Always ensure you can explain every line of code in your project, whether you wrote it or an AI helped generate it.

Last updated: March 2026 | For the Quant Analysis Platform project